Datos de fisiología del ejercicio

Laboratorio 1

Mario Morales

Contenido I

1. Lectura y preparación de los datos
2. Lectura, Limpieza e Imputación en R
3. Tratamiento de Datos Faltantes

Section 1

Lectura y preparación de los datos

Objetivo general

Enfrentar al estudiante al “dato sucio” y real.

Objetivo específico

Realizar la preparación de los datos tomados en el laboratorio para que queden listos para la realización del informe que deberán presentar los estudiantes. La preparación incluye:

- Lectura
- Limpieza
- Imputación
- Anonimización

Section 2

Lectura, Limpieza e Imputación en R

Lectura

Para esta tarea usaremos el sistema R base.

```
library(readxl)                          ①
df <- read_excel("DatosLab1.xlsx")       ②
df <- as.data.frame(df)                  ③
```

① Librería para leer archivos de Excel
② Lectura
③ Convertimos a `data.frame` puro de R Base para evitar el formato tibble

Inspección visual de los datos

```
head(df)
```

```
      Marca temporal                 Dirección de correo electrón
1 2026-03-11 12:56:49 schavescontreras80@correo.unicordoba.edu
2 2026-03-11 13:38:07       josuedramos@correo.unicordoba.edu
3 2026-03-11 13:52:10         mhlopez@correo.unicordoba.edu
4 2026-03-11 14:31:58        acrivero@correo.unicordoba.edu
5 2026-03-11 14:46:28  lpatriciaramirez@correo.unicordoba.edu
6 2026-03-11 14:52:44         vgaleano@correo.unicordoba.edu
                 Nombre Completo     Documento Fecha de Nacimien
1      Sofia Chaves Contreras  1.00037298E9          2001-06-
2   Josué David Ramos Sánchez  1.13797762E9          2007-05-
3        Mauro López Martínez 1.062434362E9          2007-02-
4 Andres Camilo Rivero Pacheco 1.067864128E9          2006-09-
5  Laura Patricia Ramírez Polo 1.071.352.721          2007-12-
6    Verónica Galeano Martínez 1.062434986E9          2007-06-
  Teléfono / Contacto Consentimiento informado diligenciado
```

Inspección visual de los datos: Columnas 6 hasta el final

```
head(df[,6:ncol(df)])
```

```
  Edad      Sexo Teléfono / Contacto Consentimiento informado
1 24.0  Femenino       3.106945186E9
2 18.0 Masculino       3.207847832E9
3 19.0 Masculino       3.206853212E9
4 19.0 Masculino        3.21358451E9
5 18.0  Femenino       3.002720373E9
6 18.0  Femenino        3.13786016E9
  Cuestionario PAR-Q diligenciado Apto para la evaluación / pr
1                              Si
2                              Si
3                              Si
4                              Si
5                              Si
6                              Si
  Restricción o condición reportada por el participante
```

Inspección visual de los datos str

```
str(df[,6:ncol(df)])
```

```
'data.frame':   37 obs. of  20 variables:
 $ Edad                                                          : chr
 $ Sexo                                                          : chr
 $ Teléfono / Contacto                                           : chr
 $ Consentimiento informado diligenciado                         : chr
 $ Cuestionario PAR-Q diligenciado                               : chr
 $ Apto para la evaluación / práctica                            : chr
 $ Restricción o condición reportada por el participante: chr
 $ Frecuencia cardíaca en reposo (lpm)                           : num
 $ Presión arterial sistólica (mmHg)                             : num
 $ Presión arterial diastólica  (mmHg)                           : num
 $ Estatura                                                      : chr
 $ Masa Corporal                                                 : chr
 $ Perímetro de Cintura                                          : chr
 $ IMC                                                           : chr
```

Renombrar columnas

En el Formato de registro de variables, hay nombres largos como `Frecuencia cardíaca en reposo`. En R base es una buena práctica:

- Evitar espacios: Usar guiones bajos (_).
- Evitar tildes y eñes: Para prevenir problemas de codificación (encoding).
- Minúsculas: R es sensible a mayúsculas (case-sensitive), y escribir todo en minúsculas evita errores comunes de digitación.

Recuperar los nombres actuales

```
colnames(df)
```

```
 [1] "Marca temporal"
 [2] "Dirección de correo electrónico"
 [3] "Nombre Completo"
 [4] "Documento"
 [5] "Fecha de Nacimiento"
 [6] "Edad"
 [7] "Sexo"
 [8] "Teléfono / Contacto"
 [9] "Consentimiento informado diligenciado"
[10] "Cuestionario PAR-Q diligenciado"
[11] "Apto para la evaluación / práctica"
[12] "Restricción o condición reportada por el participante"
[13] "Frecuencia cardíaca en reposo (lpm)"
[14] "Presión arterial sistólica (mmHg)"
[15] "Presión arterial diastólica (mmHg)"
```

Asignar nuevos nombres

Creamos un vector con los nuevos nombres

```
nombres=c("marca_temp","email","nombre","documento","f_nac", "
```

Asignar nuevos nombres

```
colnames(df)=nombres
```

Imprimir los nuevos nombres

```
colnames(df)
```

```
 [1] "marca_temp"     "email"          "nombre"         "docum
 [5] "f_nac"          "edad"           "sexo"           "tel"
 [9] "consentimiento" "par_q"          "apto"           "resti
[13] "fcr"            "presion_sis"    "presion_dia"    "estat
[17] "masa_corporal"  "cintura"        "imc"            "masa_
[21] "masa_muscular"  "masa_osea"      "grasa_vis"      "fuerz
[25] "fuerza_mi"
```

Un vistazo a estatura

```
estat_sucio=df$estatura
estat_sucio
```

```
 [1] "156.4 cm" "182.0"    "166.0"    "180.5"    "157.5"    "1
 [7] "172.3"    "164.5"    "181.0"    "168.6"    "177.0"    "1
[13] "175,5 cm" "1.7"      "163.4"    "175.5"    "182.5"    "1
[19] "174.2"    "1.7"      "156.5"    "165 cm"   "157.2 cm" "1
[25] "189.5"    "1.57"     "167.5"    "158.4"    "165.5"    "1
[31] "172.5"    "164.7"    "160.8"    "160.0"    "178.4"    "1
[37] "163.9"
```

Limpieza de "Datos Contaminados" (Uso de Expresiones Regulares)

Para limpiar el "60.3 kg" usaremos `gsub` para eliminar texto y `as.numeric` para transformar.

```
# Limpieza de la columna Edad
# gsub("[^0-9.]", "", x) elimina todo lo que NO sea número o p
df$edad <- gsub("[^0-9.]", "", df$edad)
df$edad <- as.numeric(df$edad)
# Aplicamos lo mismo a estatura y masa corporal
df$estatura <- as.numeric(gsub("[^0-9.]", "", df$estatura))
df$masa_corporal <- as.numeric(gsub("[^0-9.]", "", df$masa_cor
```

Continue usted

Haga lo mismo con las columnas que faltan

```
'data.frame':   37 obs. of  14 variables:
 $ edad          : num  24 18 19 19 18 18 18 20 18 19 ...
 $ fcr           : num  85 75 72 88 74 78 112 78 54 85 ...
 $ presion_sis   : num  122 125 113 108 110 105 137 118 124 135
 $ presion_dia   : num  90 82 69 71 70 67 33 46 82 64 ...
 $ estatura      : num  156 182 166 180 158 ...
 $ masa_corporal: num  60.3 90.7 54.3 69 52 55.2 67.3 51.5 66
 $ cintura       : num  73.4 89.4 70.5 73 67.8 70 79 68 73.9 82
 $ imc           : chr  "24.7 kg/m^2" "27.4" "19.8" "21.2" ...
 $ masa_grasa    : num  32.8 25.2 15.9 17.8 30.6 27.6 19.4 14.4
 $ masa_muscular: num  38.4 22.9 43.3 53.9 34.2 38 51.5 41.8 4
 $ masa_osea     : num  2.1 3.4 2.3 2.9 1.9 2 2.7 2.3 2.6 2.7
 $ grasa_vis     : num  3 6 1 2 1 1 2.5 1 1 5 ...
 $ fuerza_md     : num  28 32 30 38 24 28 40 36 42 42 ...
 $ fuerza_mi     : num  26 34 28 40 24 22 36 34 42 48
```

Estatura luego de la limpieza

	EstSucia	EstLimpia
10	168.6	168.6
11	177.0	177.0
12	163,2	1632.0
13	175,5 cm	1755.0
14	1.7	1.7
15	163.4	163.4

- Observe, en la fila 12, que `163,2` se transformó en 1632, lo cual está mal. Algo similar ocurre en la fila 13, con `175,5`, el cual se transforma en 1755.
- Ese error es debido a la coma `(,)`

Comas por puntos

- Primero hay que reemplazar las comas por punto

```
estat_sucio2 <- gsub(",", ".", estat_sucio)
df_est=data.frame(EstSucia1=estat_sucio, EstSucia2=estat_sucio
df_est[10:15,]
```

```
   EstSucia1 EstSucia2
10     168.6     168.6
11     177.0     177.0
12     163,2     163.2
13  175,5 cm  175.5 cm
14       1.7       1.7
15     163.4     163.4
```

- Observe que ya no hay comas (,)

Ahora si usamos la expresion regular

```
estat_fin <- as.numeric(gsub("[^0-9.]", "", estat_sucio2))
df_est=data.frame(EstSucia1=estat_sucio2, EstFinal=estat_fin)
df_est[10:15,]
```

```
   EstSucia1 EstFinal
10     168.6    168.6
11     177.0    177.0
12     163.2    163.2
13  175.5 cm    175.5
14       1.7      1.7
15     163.4    163.4
```

- Observe que ya no aparecen las unidades.

Agregamos estatura limpia al data

```
df$estatura=estat_fin
```

Un vistazo a las columnas

```
      edad            fcr          presion_sis     presion_dia
 Min.   :17.0   Min.   : 54.00   Min.   : 93.0   Min.   : 33.0
 1st Qu.:18.0   1st Qu.: 74.00   1st Qu.:110.0   1st Qu.: 66.0
 Median :18.0   Median : 79.00   Median :122.0   Median : 71.0
 Mean   :19.7   Mean   : 81.95   Mean   :120.8   Mean   : 71.4
 3rd Qu.:19.0   3rd Qu.: 88.00   3rd Qu.:129.0   3rd Qu.: 78.0
 Max.   :60.0   Max.   :117.00   Max.   :162.0   Max.   :102.0

    estatura      masa_corporal      cintura           imc
 Min.   :  1.57   Min.   : 1.80   Min.   : 56.40   Length:37
 1st Qu.:157.50   1st Qu.:51.70   1st Qu.: 69.60   Class :char
 Median :165.00   Median :62.60   Median : 73.00   Mode  :char
 Mean   :150.57   Mean   :62.03   Mean   : 90.89
 3rd Qu.:175.50   3rd Qu.:70.00   3rd Qu.: 82.00
 Max.   :189.50   Max.   :97.00   Max.   :686.00
```

Identificar filas con datos erróneos

- Quién escribió `estatura=1.57` ?

```
df[df$estatura==min(df$estatura),c("nombre","estatura")]
```

```
                nombre estatura
26 Amalfi Nuñez Doria     1.57
```

Corregir datos erróneos

- La estatura

```
df$estatura[df$estatura==min(df$estatura)]=157
summary(df$estatura)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    1.7   157.5   165.0   154.8   175.5   189.5
```

Visualizar todas las estaturas

```
df$estatura
```

```
 [1] 156.40 182.00 166.00 180.50 157.50 156.60 172.30 164.50 1
[11] 177.00 163.20 175.50   1.70 163.40 175.50 182.50 155.20 1
[21] 156.50 165.00 157.20 181.00 189.50 157.00 167.50 158.40 1
[31] 172.50 164.70 160.80 160.00 178.40   1.72 163.90
```

Una forma más eficiente para estatura

! Important

Al parecer lo que ocurre es que esas personas ingresaron la estatura en metro, entonces una forma de corregir este problema de manera rápida y automática es multiplicando esos valores por 100.

```
df$estatura[df$estatura<2.2]=df$estatura[df$estatura<2.2]*100
summary(df$estatura)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  155.2   160.8   167.5   168.5   175.5   189.5
```

Un detalle con imc

```
cbind(df$imc,df$imcn)
```

```
      [,1]           [,2]
 [1,] "24.7 kg/m^2" "24.72"
 [2,] "27.4"        "27.4"
 [3,] "19.8"        "19.8"
 [4,] "21.2"        "21.2"
 [5,] "21.0"        "21"
 [6,] "22.5"        "22.5"
 [7,] "22.7"        "22.7"
 [8,] "19.1"        "19.1"
 [9,] "20.1"        "20.1"
[10,] "24.6"        "24.6"
[11,] "20.1"        "20.1"
[12,] "17,9"        NA
[13,] "20,6 kg/m2"  NA
[14,] "NA"          NA
```

Corregimos imc

```
# 1. Cambiamos la coma por punto (para que R entienda decimale
imcn <- gsub(",", ".", df$imc)

cbind(df$imc,imcn)
```

```
                      imcn
 [1,] "24.7 kg/m^2" "24.7 kg/m^2"
 [2,] "27.4"        "27.4"
 [3,] "19.8"        "19.8"
 [4,] "21.2"        "21.2"
 [5,] "21.0"        "21.0"
 [6,] "22.5"        "22.5"
 [7,] "22.7"        "22.7"
 [8,] "19.1"        "19.1"
 [9,] "20.1"        "20.1"
[10,] "24.6"        "24.6"
[11,] "20.1"        "20.1"
```

Corregimos imc II

```
# 2. Usamos sub() con una expresión que busca el primer bloque
# Explicación del patrón:
# ^[^0-9]* -> Ignora todo al inicio que NO sea un número
# ([0-9]+\\.?[0-9]*) -> CAPTURA el primer número (con su decim
# .*$      -> Ignora todo lo que venga después (como " kg/m^2
imcn <- sub("^[^0-9]*([0-9]+\\.?[0-9]*).*$", "\\1", imcn)

#  3. Finalmente convertimos a numérico
imcn <- as.numeric(imcn)
data.frame(df$imc,imcn)
```

```
      df.imc  imcn
1  24.7 kg/m^2  24.7
2         27.4  27.4
3         19.8  19.8
4         21.2  21.2
5         21.0  21.0
```

Agregamos imc a df

```
df$imc=imcn
```

Inspección rápida de las variables

```
summary(df[,nombres2])
```

```
      edad            fcr          presion_sis     presion_dia
 Min.   :17.0   Min.   : 54.00   Min.   : 93.0   Min.   : 33.0
 1st Qu.:18.0   1st Qu.: 74.00   1st Qu.:110.0   1st Qu.: 66.0
 Median :18.0   Median : 79.00   Median :122.0   Median : 71.0
 Mean   :19.7   Mean   : 81.95   Mean   :120.8   Mean   : 71.4
 3rd Qu.:19.0   3rd Qu.: 88.00   3rd Qu.:129.0   3rd Qu.: 78.0
 Max.   :60.0   Max.   :117.00   Max.   :162.0   Max.   :102.0

    estatura     masa_corporal      cintura           imc
 Min.   :155.2   Min.   : 1.80   Min.   : 56.40   Min.   :  0.
 1st Qu.:160.8   1st Qu.:51.70   1st Qu.: 69.60   1st Qu.: 19.
 Median :167.5   Median :62.60   Median : 73.00   Median : 21.
 Mean   :168.5   Mean   :62.03   Mean   : 90.89   Mean   : 28.
 3rd Qu.:175.5   3rd Qu.:70.00   3rd Qu.: 82.00   3rd Qu.: 24.
 M      :180.5   M      :97.00   M      :626.00   M      :253.
```

Identificar filas con datos erróneos

- Quién escribió `imc=253.1` ?

```
df[df$imc==max(df$imc,na.rm=TRUE),c("nombre","imc")]
```

```
                        nombre   imc
NA                         <NA>    NA
NA.1                       <NA>    NA
NA.2                       <NA>    NA
NA.3                       <NA>    NA
36   Angel Josep Ramos Paez 253.1
```

Corregir `imc`

Corregir datos erróneos imc

- El imc

```
df$imc[df$imc==max(df$imc,na.rm=TRUE)]=25
summary(df$imc)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
   0.00   19.10   21.00   21.81   24.90   33.50       4
```

Reemplazar los ceros por NA

! Important

Sabemos que los ceros son de los estudiantes menores de edad que el instrumento de medición no entregó el dato, por lo tanto es un dato faltante. Reemplazaremos los ceros por NA porque luego haremos imputación de datos faltantes.

```
df$imc[df$imc==0]=NA
summary(df$imc)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
  16.80   19.25   21.10   22.49   24.93   33.50       5
```

Reemplazar los ceros por NA

```
df$masa_grasa[df$masa_grasa==0]=NA
df$masa_muscular[df$masa_muscular==0]=NA
df$masa_osea[df$masa_osea==0]=NA
df$grasa_vis[df$grasa_vis==0]=NA
```

Inspección rápida de las variables

```
summary(df[,nombres2])
```

```
      edad            fcr          presion_sis     presion_dia
 Min.   :17.0   Min.   : 54.00   Min.   : 93.0   Min.   : 33.0
 1st Qu.:18.0   1st Qu.: 74.00   1st Qu.:110.0   1st Qu.: 66.0
 Median :18.0   Median : 79.00   Median :122.0   Median : 71.0
 Mean   :19.7   Mean   : 81.95   Mean   :120.8   Mean   : 71.4
 3rd Qu.:19.0   3rd Qu.: 88.00   3rd Qu.:129.0   3rd Qu.: 78.0
 Max.   :60.0   Max.   :117.00   Max.   :162.0   Max.   :102.0

    estatura     masa_corporal      cintura           imc
 Min.   :155.2   Min.   : 1.80   Min.   : 56.40   Min.   :16.8
 1st Qu.:160.8   1st Qu.:51.70   1st Qu.: 69.60   1st Qu.:19.2
 Median :167.5   Median :62.60   Median : 73.00   Median :21.1
 Mean   :168.5   Mean   :62.03   Mean   : 90.89   Mean   :22.4
 3rd Qu.:175.5   3rd Qu.:70.00   3rd Qu.: 82.00   3rd Qu.:24.9
```

Ejercicios

- Identifique a la persona que escribió `cintura=686` .
- Corrija de acuerdo con el dato real que debe tener el estudiante en su formulario en papel.
- Identifique a la persona que escribió `presion_sis=162` .
- Corrija de acuerdo con el dato real que debe tener el estudiante en su formulario en papel.
- En caso que ese valor esté correcto debe repetir la medición y si persiste se debe remitir al médico.

Anonimización de Datos

Manipularemos las columnas por índice o nombre y a generar identificadores únicos.

```
# 1. Identificar columnas sensibles por nombre
columnas_sensibles <- c("marca_temp", "email", "nombre","docum

# 2. Crear un ID aleatorio único para cada fila
set.seed(123)
df$id_estudiante <- sample(1000:9999, nrow(df), replace = FALS
```

Anonimización de Datos

Manipularemos las columnas por índice o nombre y a generar identificadores únicos.

```
# 3. Crear el nuevo dataframe omitiendo las sensibles
# Usamos la indexación negativa o selección por nombre
df_anonimo <- subset(df, select= !(names(df) %in% columnas_ser

# 4. Reordenar para que el ID sea la primera columna
df_anonimo <- df_anonimo[, c(ncol(df_anonimo), 1:(ncol(df_anor
```

Columnas del df anónimo

```
colnames(df_anonimo)
```

```
 [1] "id_estudiante"  "edad"           "consentimiento" "par_c
 [5] "apto"           "restriccion"    "fcr"            "presi
 [9] "presion_dia"    "estatura"       "masa_corporal"  "cintu
[13] "imc"            "masa_grasa"     "masa_muscular"  "masa_
[17] "grasa_vis"      "fuerza_md"      "fuerza_mi"      "imcn"
[21] "imcn2"
```

Volver a unir las columnas sensibles

Para eso hay que dejar un `data frame` con las columnas sensibles y la columna `id_estudiante` que sirve como `llave`

```
# Creamos un dataframe aparte que SOLO tenga la llave y los no
df_llave_maestra <- df[, c("id_estudiante","marca_temp", "emai
```

Datos desordenados

Simulemos que alguien desordena las filas del `data frame` anónimo por completo:

```
# Desordenamos las filas aleatoriamente
df_anonimo_desordenado <- df_anonimo[sample(nrow(df_anonimo)),
```

Juntemos

Ahora juntamos la información sensible al data después de desordenado

```
# Unimos los nombres de vuelta al archivo desordenado usando l
df_recuperado <- merge(x = df_anonimo_desordenado,
                       y = df_llave_maestra,
                       by = "id_estudiante")
```

Custodia de datos

- El archivo que usarán para el informe no tendrá los nombres, solo el `id_estudiante`.
- Si alguno quiere saber de quién es un dato (por ejemplo, quién tiene la mayor fuerza prensil), tendrá que pedírselo al "dueño de la llave"
- Este es el concepto de custodia de datos.

Section 3

Tratamiento de Datos Faltantes

Identificación de Datos Faltantes

- Los estudiantes menores de 18 años, no tienen los datos arrojados por la balanza.
- Primero identificaremos las columnas que tienen datos faltantes, y contaremos cuántos hay

```
# Retorna TRUE si hay al menos un dato faltante en cualquier p
any(is.na(df_anonimo))
```

```
[1] TRUE
```

```
# Cuenta el total de datos faltantes en toda la matriz
sum(is.na(df_anonimo))
```

```
[1] 36
```

Identificación por columnas

Es más útil identificar los faltantes por columnas

Podemos usar la función `colSums()` combinada con `is.na()`. Esto nos dará una lista clara de qué variables necesitan atención (como la grasa visceral o la masa ósea en los menores de 18 años).

```
# Retorna el número de NAs por cada columna
conteo_nas <- colSums(is.na(df_anonimo))
print(conteo_nas)
```

```
 id_estudiante          edad consentimiento           par_q
             0             0              0               0
   restriccion           fcr    presion_sis    presion_dia
             0             0              0               0
 masa_corporal       cintura            imc     masa_grasa  m
             0             0              5               6
     masa_osea      grasa_vis     fuerza_md      fuerza_mi
             5             5              0               0
```

Qué columnas tienen NA's

La siguiente función nos dice qué columnas tienen datos faltantes

```
cual_na=function(df){
  cual=apply(apply(df,2,is.na),2,any)
  colnames(df)[cual]
}
vars_na=cual_na(df_anonimo)
vars_na
```

```
[1] "imc"           "masa_grasa"    "masa_muscular" "masa_osea
[5] "grasa_vis"     "imcn"          "imcn2"
```

Cuántos NA's hay por columna

```
cols=cual_na(df_anonimo)
apply(apply(df[,cols],2,is.na),2,sum)
```

```
            imc     masa_grasa masa_muscular      masa_osea             gr
              5              6             5              5
           imcn          imcn2
              6              4
```

Imputación de faltantes

Luego de identificar las variables de composición con `NA's`, procedemos a imputar, estimando el dato faltante usando la mediana.

```
for(v in vars_na) {
  # Calcular la mediana ignorando los NAs
  mediana_v <- median(df_anonimo[, v], na.rm = TRUE)

# Reemplazar solo donde hay NA
df_anonimo[is.na(df_anonimo[, v]), v] <- mediana_v
}
```